

Koopman–EDMD + MPC for Lateral Vehicle Control

Sanskar Sunil Patil (1234434658)
EEE 598: Data-driven Dynamical Systems
Arizona State University
spati179@asu.edu

Abstract—Data-driven lateral vehicle control is studied using a simulated dynamic bicycle model formulated in road-aligned tracking-error coordinates. The project develops a reproducible pipeline that generates informative simulation data, learns a Koopman-based Extended Dynamic Mode Decomposition (EDMD) lifted linear predictor, and embeds that predictor into a constrained Model Predictive Control (MPC) framework for steering control. The learned Koopman/EDMD model achieves extremely accurate one-step and finite-horizon open-loop prediction in the nominal operating regime, and the resulting Koopman-MPC remains feasible and stable under nominal lane-change and slalom maneuvers. However, closed-loop evaluation shows that Koopman-MPC does not outperform the classical feedforward plus LQR baseline, which achieves lower tracking error and lower steering effort. Robustness studies across speed further show that the fixed-speed Koopman controller degrades severely at high speed, while a speed-aware EDMD extension still fails to surpass the baseline. Overall, the project shows that although SINDy is the strongest learned predictor in open loop, the feedforward plus LQR controller is the strongest controller in closed loop, highlighting that the most accurate predictor is not necessarily the best controller.

I. INTRODUCTION AND MOTIVATION

Accurate lateral vehicle control is a core requirement in autonomous driving and advanced driver-assistance systems because the vehicle must follow a desired lane centerline or reference path while maintaining stability, respecting actuator limits, and remaining robust across different maneuvers and operating conditions. Even in simulation, this problem captures the essential engineering trade-offs of real path-tracking systems: the controller must reduce lateral and heading error, avoid excessive steering activity, and remain reliable when the reference curvature or vehicle speed changes. These requirements make lateral control an appropriate and practically relevant benchmark for studying modern data-driven modeling and control methods.

A standard engineering approach to this problem is to begin with a physics-based vehicle model, derive a tractable control-oriented approximation, and then design a feedback controller such as LQR or MPC around that model. In this project, that classical viewpoint is represented by a dynamic bicycle model in road-aligned tracking-error coordinates together with a feedforward plus LQR baseline controller. This baseline is important because it provides a strong, interpretable reference point: it uses established control structure, is computationally simple, and performs well when the operating assumptions are

well matched to the system. At the same time, classical model-based approaches often require operating-point assumptions, careful controller tuning, and additional design effort when the goal is to handle broader operating envelopes or to integrate learned dynamics directly into an optimization-based control framework.

Data-driven dynamical-systems methods offer an appealing alternative because they aim to learn predictive models directly from trajectories generated by the system. Among these methods, Koopman-based Extended Dynamic Mode Decomposition (EDMD) is especially attractive for control because it seeks a lifted representation in which the system evolves approximately linearly in a higher-dimensional observable space. If such a lifted linear predictor is accurate enough, it can be embedded naturally inside a constrained Model Predictive Control formulation. This gives Koopman/EDMD a particularly strong engineering appeal: it promises a bridge between nonlinear system behavior and convex optimization-based control design. In parallel, sparse-regression approaches such as SINDy are attractive because they can often recover compact, highly accurate predictors when the underlying dynamics are well represented by the chosen feature library. These two methods therefore motivate a deeper question than simple prediction accuracy alone. The real issue is not only which learned model predicts the truth model most accurately, but which learned representation is most useful when placed inside a closed-loop controller.

This project studies that question using a fully reproducible end-to-end pipeline for lateral vehicle modeling and control. The ground-truth system is a simulated dynamic bicycle model with linear tire forces under a small-slip approximation, expressed in road-aligned error coordinates so that the state directly represents lateral tracking error, heading error, lateral velocity, and yaw rate. Using this truth model, the project generates informative training, validation, and test trajectories under multiple maneuvers, including straight driving, lane-change, and slalom scenarios, with persistently exciting steering inputs and curvature-based references. A Koopman/EDMD lifted linear predictor is then learned from these rollouts in standardized coordinates, with steering and reference curvature included as inputs so that changing road geometry can be represented within the learned predictor. That lifted model is subsequently used inside a constrained MPC formulation with steering and steering-rate limits. To place the learned

controller in context, the report compares it against a classical feedforward plus LQR baseline and also evaluates SINDy as a competing learned predictor. In this sense, the report is intentionally comparative rather than purely demonstrative: it studies the full chain from simulated dynamics and dataset generation to prediction quality, controller synthesis, closed-loop evaluation, and robustness analysis.

The motivation for this comparative structure is that strong open-loop prediction and strong closed-loop control are not necessarily the same achievement. A learned model may have very small one-step or finite-horizon rollout error and still perform poorly when placed inside a controller, especially if the learned structure is only weakly robust to distribution shift or operating-point variation. Conversely, a simpler classical model-controller pair may deliver stronger and more reliable tracking despite being less sophisticated from a data-driven modeling perspective. This project therefore evaluates not only nominal tracking performance, but also robustness across maneuvers and across longitudinal speeds. The speed-sweep experiments are particularly important because they expose whether a controller learned around one operating condition remains reliable as the plant changes. That question is central to practical vehicle control, where speed variation is unavoidable and often changes the effective dynamics substantially.

The main engineering conclusion of the project is both clear and instructive. Koopman/EDMD produces a controller-friendly lifted linear model and achieves extremely strong nominal open-loop prediction in the simulated operating regime, but the resulting Koopman-MPC does not outperform the classical feedforward plus LQR baseline in closed-loop tracking. Across lane-change and slalom maneuvers, the baseline yields smaller lateral-tracking errors and lower steering effort, and across a speed sweep the fixed-speed Koopman controller degrades severely at high speed. A speed-aware EDMD extension was also explored, but it still failed to recover robust closed-loop performance. At the same time, SINDy emerges as the strongest learned predictor in open-loop testing. The central message of the project is therefore that the best predictor is not automatically the best controller. More broadly, the report highlights a key lesson in data-driven control: prediction accuracy, controller-compatible structure, and closed-loop robustness are related but fundamentally different design objectives, and successful engineering requires evaluating all three rather than assuming that one implies the others.

II. SYSTEM DESCRIPTION (MATHEMATICAL MODEL)

A. Ground-truth lateral vehicle dynamics

The ground-truth plant used throughout this project is a planar dynamic bicycle model written in road-aligned tracking-error coordinates. This model is used for data generation, model identification, and controller evaluation. The state is defined as

$$\mathbf{x} = [e_y \quad e_\psi \quad v_y \quad r]^\top, \quad (1)$$

where e_y denotes the lateral tracking error relative to the reference path, e_ψ denotes the heading error relative to the reference heading, v_y is the lateral velocity, and r is the yaw rate. The physical control input is the front steering angle δ . The longitudinal speed V_x and the reference curvature $\kappa_{\text{ref}}(t)$ are treated as known exogenous signals. In the notebook implementation, V_x is held constant over each rollout, while $\kappa_{\text{ref}}(t)$ varies with the selected maneuver.

The continuous-time vehicle dynamics are based on the standard small-slip dynamic bicycle approximation with linear tire forces,

$$F_{yf} = C_{af}\alpha_f, \quad F_{yr} = C_{ar}\alpha_r, \quad (2)$$

where C_{af} and C_{ar} are the front and rear cornering stiffnesses, respectively, and α_f, α_r are the front and rear slip angles. Under the small-angle approximation used in the notebook,

$$\alpha_f \approx \delta - \frac{v_y + l_f r}{V_x}, \quad \alpha_r \approx -\frac{v_y - l_r r}{V_x}. \quad (3)$$

With these assumptions, the lateral and yaw dynamics are

$$\dot{v}_y = \frac{F_{yf} + F_{yr}}{m} - V_x r, \quad (4)$$

$$\dot{r} = \frac{l_f F_{yf} - l_r F_{yr}}{I_z}, \quad (5)$$

and the road-aligned tracking-error kinematics are

$$\dot{e}_y = v_y + V_x e_\psi, \quad (6)$$

$$\dot{e}_\psi = r - V_x \kappa_{\text{ref}}(t). \quad (7)$$

Substituting the linear tire-force expressions yields the explicit state-space form

$$\dot{e}_y = v_y + V_x e_\psi, \quad (8)$$

$$\dot{e}_\psi = r - V_x \kappa_{\text{ref}}, \quad (9)$$

$$\dot{v}_y = -\frac{C_{af} + C_{ar}}{mV_x} v_y - \left(\frac{C_{af}l_f - C_{ar}l_r}{mV_x} + V_x \right) r + \frac{C_{af}}{m} \delta, \quad (10)$$

$$\dot{r} = -\frac{C_{af}l_f - C_{ar}l_r}{I_z V_x} v_y - \frac{C_{af}l_f^2 + C_{ar}l_r^2}{I_z V_x} r + \frac{C_{af}l_f}{I_z} \delta. \quad (11)$$

This model serves as the reference plant for all simulations and evaluations.

B. Discrete-time simulation model

Although the vehicle model is introduced in continuous time, all simulation and data generation in the notebook are carried out in discrete time through numerical integration. The implemented simulator uses a forward-Euler update of the form

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t \dot{\mathbf{x}}_k, \quad (12)$$

with $\dot{\mathbf{x}}_k$ evaluated from the dynamic bicycle equations above. This discrete simulator is implemented in the notebook as the function `bike_step(x, delta, kappa_ref, Vx, cfg)` and is repeatedly applied to produce open-loop rollouts and closed-loop trajectories. The corresponding rollout

function is then used to generate identification data and to evaluate controller behavior under different maneuver scenarios.

A small numerical safeguard is also included in the implementation. Because the slip-angle formulas contain division by V_x , the code clamps the effective longitudinal speed below by a small positive value to avoid division by zero or near-singular behavior. In the notebook, the effective speed is taken as

$$V_{x,\text{eff}} = \max(0.1, V_x), \quad (13)$$

which preserves the intended dynamics while preventing numerical failure in simulation. This is an implementation-level assumption rather than a change to the model itself, but it is important to report because it affects robustness of the simulator.

C. Nominal parameters and assumptions

The notebook uses a fixed configuration object that stores the nominal vehicle, simulation, and control parameters. The nominal vehicle parameters are

$$m = 1500 \text{ kg}, \quad I_z = 3000 \text{ kg m}^2, \quad (14)$$

$$l_f = 1.2 \text{ m}, \quad l_r = 1.6 \text{ m}, \quad (15)$$

$$C_{af} = 80,000 \text{ N/rad}, \quad C_{ar} = 80,000 \text{ N/rad}. \quad (16)$$

The nominal operating speed is

$$V_x = 15 \text{ m/s}, \quad (17)$$

and the simulation settings are

$$t_0 = 0, \quad t_f = 25 \text{ s}, \quad \Delta t = 0.01 \text{ s}. \quad (18)$$

The steering actuator is constrained by both amplitude and rate limits,

$$|\delta| \leq 0.5 \text{ rad}, \quad |\dot{\delta}| \leq 0.8 \text{ rad/s}. \quad (19)$$

For the nominal MPC design, the prediction horizon is set to 2.0 s, which corresponds to

$$N = \frac{2.0}{0.01} = 200 \quad (20)$$

prediction steps at the nominal sampling time. These values are taken directly from the notebook configuration and define the baseline operating regime for the learned model and controller studies.

The project assumes a deterministic, noise-free simulation environment. No measurement noise, sensor corruption, or external disturbances are added to the truth-model trajectories used for training and validation. As a result, the identification problem is intentionally clean, which makes it easier to isolate differences among modeling approaches such as EDMD and SINDy. This assumption is important because it partly explains why the learned predictors can achieve extremely small one-step prediction errors in the nominal operating regime.

D. Reference maneuvers and curvature-based path generation

Rather than prescribing reference trajectories directly in Cartesian coordinates, the notebook generates maneuvers through a curvature profile $\kappa_{\text{ref}}(t)$. This is a natural choice because the road-aligned error dynamics depend directly on reference curvature, and it provides a compact way to define multiple path shapes within a unified framework. Given longitudinal speed V_x and reference curvature $\kappa_{\text{ref}}(t)$, the notebook computes the reference yaw rate, heading, and planar reference path by numerical integration:

$$r_{\text{ref}}(t) = V_x \kappa_{\text{ref}}(t), \quad (21)$$

$$\psi_{\text{ref}}(t) = \int r_{\text{ref}}(t) dt, \quad (22)$$

$$x_{\text{ref}}(t) = \int V_x \cos(\psi_{\text{ref}}(t)) dt, \quad (23)$$

$$y_{\text{ref}}(t) = \int V_x \sin(\psi_{\text{ref}}(t)) dt. \quad (24)$$

This curvature-based construction produces smooth maneuver families while keeping the exogenous input entering the tracking-error dynamics explicit.

Three maneuver classes are used throughout the project:

- 1) **Straight driving**, for which $\kappa_{\text{ref}}(t) = 0$;
- 2) **Lane change**, modeled as two smooth curvature pulses of opposite sign, producing an S-shaped reference path with near-zero final heading change;
- 3) **Slalom**, modeled as a sinusoidal curvature signal.

In the reference-generation code, the lane-change maneuver is implemented using smooth pulse functions, while the slalom maneuver is implemented using a sinusoidal curvature profile. For the illustrative nominal case shown in the notebook, the selected maneuver is a lane change with

$$\kappa_{\text{max}} = 0.002 \text{ m}^{-1}, \quad (25)$$

which produces a final reference displacement of approximately $(x_{\text{ref}}, y_{\text{ref}}) = (374.50, 10.78) \text{ m}$ over the simulation horizon. These reference maneuvers are reused later for data generation, controller testing, and comparative evaluation.

E. Role of the truth model in the overall project

This mathematical model is not merely background material; it is the central reference system against which all later learned models and controllers are judged. Open-loop rollouts from the truth model are used to generate the EDMD and SINDy datasets. The same simulator is then used as the plant in closed-loop experiments for the baseline controller, Koopman-MPC, and the speed-aware controller variants. In other words, the project deliberately separates the *truth dynamics* from the *learned prediction models*: the bicycle model defines the real plant in simulation, while the learned models are approximations constructed from data generated by that plant. This distinction is essential for interpreting the later results on prediction accuracy, controller performance, and robustness.

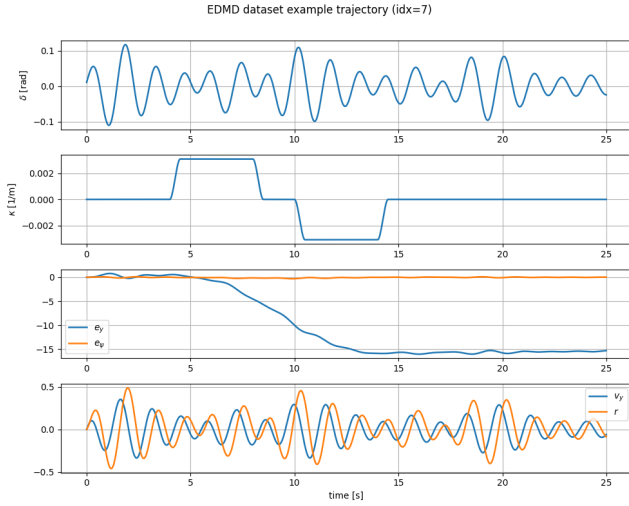


Fig. 1. Example trajectory from the EDMD training dataset. The dataset is generated using persistently exciting steering inputs and reference-curvature maneuvers so that the learned model observes diverse state and input combinations.

III. DATA DESCRIPTION

The dataset used in this project is generated entirely from simulation using the ground-truth dynamic bicycle model described in the previous section. This choice makes the study fully controlled and reproducible, since the same simulator can be used consistently for data generation, model learning, and controller evaluation. It also isolates the central question of the project—the relationship between prediction quality and control performance—without introducing additional complications such as measurement noise, sensor dropouts, missing channels, or experimental inconsistencies from real-world data collection. In this sense, the simulated dataset is not intended to mimic all practical deployment issues; rather, it is designed to provide a clean environment in which the behavior of Koopman/EDMD, SINDy, and classical control can be compared fairly.

A. Source of data and sampling

All trajectories are produced by numerically simulating the truth-model bicycle dynamics with forward-Euler integration. For the nominal fixed-speed study, the notebook uses the configuration

$$\Delta t = 0.01 \text{ s}, \quad t_0 = 0, \quad t_f = 25 \text{ s}, \quad (26)$$

so each rollout spans the same fixed simulation horizon at a uniform sampling rate. The nominal longitudinal speed for the main EDMD study is

$$V_x = 15 \text{ m/s}. \quad (27)$$

At every step, the simulator evolves the state according to the prescribed steering input and reference-curvature signal, producing full state trajectories that are later split into training, validation, and test sets. Because the project is simulation-based and deterministic, repeated runs with the same seed and configuration reproduce the same dataset exactly.

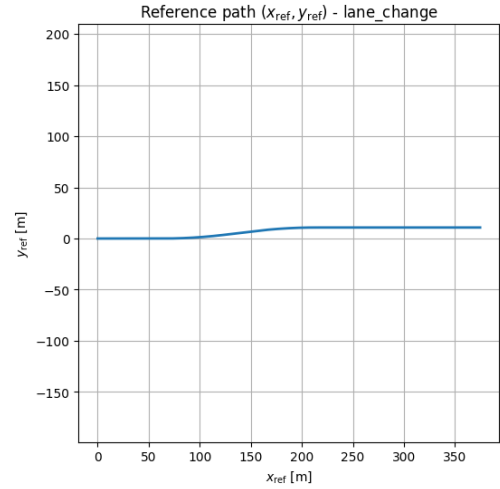


Fig. 2. Lane-change reference path used for lateral tracking experiments. The path is generated by integrating a smooth curvature profile, producing an S-shaped maneuver with near-zero final heading change.

B. Dataset construction

The quality of Koopman/EDMD depends strongly on how informative the identification data are. For that reason, the notebook does not rely on a single nominal trajectory or a single simple maneuver. Instead, it generates a collection of open-loop rollouts using persistently exciting steering inputs together with multiple reference-maneuver types. The excitation design includes multi-sine steering mixtures with randomized frequencies, phases, and weights, followed by mild smoothing and explicit enforcement of steering-rate and steering-magnitude bounds. This produces trajectories that are rich enough to probe the local dynamics of the simulated vehicle while still remaining physically reasonable under the actuator constraints.

For the nominal fixed-speed EDMD dataset, the notebook generates a total of 30 trajectories. The maneuver type alternates among three families:

- 1) straight driving,
- 2) lane-change,
- 3) slalom.

For non-straight maneuvers, the curvature amplitude is randomized over a bounded range, and the steering excitation amplitude is also randomized from rollout to rollout. This prevents the dataset from collapsing onto one narrow operating condition and improves coverage of the identification regime. For each trajectory, the notebook stores the full state rollout, the steering sequence, the curvature sequence, and a small configuration record describing the maneuver parameters used to generate that rollout.

C. Train/validation/test split

To avoid information leakage, the dataset is split by *entire trajectory* rather than by individual time sample. This is an important design choice: if adjacent samples from the same

rollout were distributed across training and validation sets, the resulting performance estimates would be overly optimistic because the learned model would effectively see nearly identical local dynamics during both fitting and evaluation. The notebook therefore uses a trajectory-level split consisting of

- 21 training trajectories,
- 4 validation trajectories,
- 5 test trajectories.

The training set is used to fit model parameters and scaling statistics, the validation set is used to assess predictive quality and tune modeling choices, and the test set is reserved for final multi-step reporting and comparison. This split is explicitly emphasized in the notebook as a more reliable measure of generalization.

D. Measured variables and stored signals

The simulated dataset records the full physical state of the truth model at each time step. The measured state variables are

$$\mathbf{x}_k = [e_{y,k} \quad e_{\psi,k} \quad v_{y,k} \quad r_k]^\top, \quad (28)$$

where e_y is lateral tracking error, e_ψ is heading error, v_y is lateral velocity, and r is yaw rate. In addition to the state, the data-generation pipeline records the steering input δ_k and the reference-curvature sequence κ_k used during the rollout. For the fixed-speed Koopman/EDMD model, the input to the learned predictor is the augmented signal

$$\mathbf{u}_k = \begin{bmatrix} \delta_k \\ \kappa_k \end{bmatrix}. \quad (29)$$

This augmentation is important because the reference curvature enters the tracking-error dynamics as an exogenous signal, especially through the heading-error equation, so the learned model must account for changing path geometry rather than treating it as unmodeled disturbance. For the later speed-aware extension, the augmented input becomes

$$\mathbf{u}_k = \begin{bmatrix} \delta_k \\ \kappa_k \\ V_x \end{bmatrix}, \quad (30)$$

so that a single learned model can attempt to represent dynamics across multiple operating speeds.

E. Preprocessing and normalization

The notebook applies preprocessing mainly for numerical conditioning rather than for denoising. Since the data are generated from deterministic simulation, there is no measurement-noise filtering stage. Instead, preprocessing focuses on two steps.

First, the Koopman/EDMD model is built in a lifted observable space, where the physical state is mapped into a vector of nonlinear features including constant, polynomial, cross, and trigonometric terms. Because different lifted coordinates can have very different magnitudes, direct regression in unscaled coordinates would be poorly conditioned.

Second, both the lifted-state matrix and the input matrix are standardized row-wise using training-set means and standard deviations. In the notebook notation,

$$z_s = \frac{z - \mu_z}{\sigma_z}, \quad u_s = \frac{u - \mu_u}{\sigma_u}. \quad (31)$$

The EDMD regression is then fit in these standardized coordinates. The constant bias feature in the lifted state is treated specially and is intentionally not standardized, so that affine capability is preserved. During prediction, the process is reversed by propagating the standardized lifted state, unscaling it, and then reconstructing the physical state through the fixed selector matrix. This preprocessing is one of the reasons the regression remains numerically stable despite the nonlinear lifted dictionary.

F. Data coverage and operating-regime limitations

An important theme of the project is that learned-model performance depends heavily on the region covered by the training data. A predictor can perform extremely well inside the operating regime represented during identification and still degrade badly when evaluated under conditions not well represented in that dataset. The notebook explicitly notes that performance may worsen when the controller is tested at speeds not seen in training, under more aggressive curvature profiles, or in operating regions requiring larger steering actions than those used during data generation. This motivates the later robustness sweeps over speed and the additional speed-aware EDMD experiment. In other words, the dataset is intentionally rich within a nominal regime, but it is not assumed to guarantee robustness outside that regime.

G. Role of the dataset in the full pipeline

The generated trajectories support several different stages of the overall project. The training subset is used to fit the Koopman/EDMD lifted linear predictor and the SINDy model. The validation subset is used to assess one-step predictive quality and error statistics. The test subset is then used for multi-step open-loop rollout evaluation to determine whether low one-step error translates into stable finite-horizon prediction. Finally, the same truth simulator that produced the dataset is reused in closed loop to compare the baseline controller, Koopman-MPC, and speed-aware variants. This makes the data section central to the entire workflow: it connects the mathematical model, the identification stage, and the final control comparisons into one consistent experimental pipeline.

IV. METHODOLOGY

A. Overall workflow and chosen approach

The baseline vehicle model implementation is adapted from [1]. The pipeline uses Koopman-based EDMD for modeling and MPC for control. The central idea is to replace direct nonlinear state prediction with a lifted linear predictor that is more convenient for optimization-based control. The notebook is organized as a fully reproducible end-to-end workflow in which the same truth-model simulator is used for data

generation, learned-model fitting, and controller evaluation. The complete pipeline is

- 1) define the ground-truth lateral vehicle model in road-aligned error coordinates,
- 2) generate informative open-loop rollouts using persistently exciting steering inputs and multiple maneuver families,
- 3) construct a nonlinear lifting map for the physical state,
- 4) identify a linear lifted predictor using EDMD in standardized coordinates,
- 5) embed the learned lifted model into a constrained finite-horizon MPC controller,
- 6) evaluate prediction accuracy in one-step and multi-step settings,
- 7) test the resulting controller in closed loop under nominal maneuvers and robustness sweeps,
- 8) compare the learned-model approach against a classical feedforward plus LQR baseline and against a SINDy predictor.

This structure studies the trade-off among prediction accuracy, model structure, and closed-loop performance.

A classical feedforward plus LQR controller is retained throughout the study as the main control benchmark. This is important because the objective is not to show that a learned method can be made to work in isolation, but to assess whether it improves on a strong and interpretable baseline. In parallel, SINDy is included as a predictor-comparison benchmark so that the report can distinguish between *best predictor* and *best controller*.

B. Koopman/EDMD lifted modeling idea

Consider the discrete-time nonlinear dynamics

$$\mathbf{x}_{k+1} = F(\mathbf{x}_k, \mathbf{u}_k), \quad (32)$$

where $\mathbf{x}_k \in \mathbb{R}^4$ is the physical state and $\mathbf{u}_k$ is the input. Instead of evolving the physical state directly, the Koopman/EDMD approach defines a lifted observable vector

$$\mathbf{z}_k = \psi(\mathbf{x}_k) \in \mathbb{R}^{n_z}, \quad (33)$$

and approximates the lifted dynamics by a linear system of the form

$$\mathbf{z}_{k+1} \approx A\mathbf{z}_k + B\mathbf{u}_k. \quad (34)$$

If the observable dictionary is sufficiently expressive, nonlinear dynamics can be approximated as linear evolution in lifted coordinates. This is especially attractive for controller design because linear prediction models can be embedded directly into convex quadratic MPC formulations.

For the fixed-speed EDMD model, the notebook uses an augmented input

$$\mathbf{u}_k = \begin{bmatrix} \delta_k \\ \kappa_{\text{ref},k} \end{bmatrix}, \quad (35)$$

where δ_k is steering and $\kappa_{\text{ref},k}$ is the reference curvature preview. Including curvature as part of the input is essential because the heading-error dynamics contain the exogenous

term $-V_x \kappa_{\text{ref}}(t)$, so a learned predictor that ignores curvature would be forced to absorb changing road geometry into model error. For the speed-aware extension, the augmented input is expanded to

$$\mathbf{u}_k = \begin{bmatrix} \delta_k \\ \kappa_{\text{ref},k} \\ V_x \end{bmatrix}, \quad (36)$$

so that one model can attempt to represent multiple operating speeds.

C. Lifting dictionary and exact state reconstruction

The lifting dictionary used in the notebook explicitly contains the physical state variables together with polynomial, cross, and trigonometric features:

$$\psi(\mathbf{x}) = [1, e_y, e_\psi, v_y, r, e_y^2, e_\psi^2, v_y^2, r^2, e_y e_\psi, e_y v_y, e_y r, e_\psi v_y, e_\psi r, v_y r, \sin(e_\psi), \cos(e_\psi)]^\top. \quad (37)$$

This gives a lifted dimension of

$$n_z = 17. \quad (38)$$

A useful design choice in this project is that the physical state is embedded directly inside the lifted vector. As a result, state reconstruction is exact rather than learned. Specifically, there exists a fixed selector matrix C_{sel} such that

$$\mathbf{x}_k = C_{\text{sel}} \mathbf{z}_k. \quad (39)$$

This is convenient because it allows the controller and evaluation code to recover the physical variables from the lifted coordinates without introducing an additional learned decoder. In other words, the only learned component is the lifted state-transition law; state extraction remains structurally exact.

D. EDMD identification in standardized coordinates

For numerical conditioning, the notebook does not fit the EDMD model directly in raw lifted coordinates. Instead, both the lifted-state samples and input samples are standardized row-wise using training-set means and standard deviations:

$$\mathbf{z}_s = \frac{\mathbf{z} - \mu_z}{\sigma_z}, \quad \mathbf{u}_s = \frac{\mathbf{u} - \mu_u}{\sigma_u}. \quad (40)$$

The constant bias feature in the lifted state is handled specially so that it is not distorted by standardization. In these standardized coordinates, the fixed-speed lifted predictor is

$$\mathbf{z}_{s,k+1} \approx A\mathbf{z}_{s,k} + B\mathbf{u}_{s,k}. \quad (41)$$

The training data are assembled from stacked trajectory samples into matrices

$$Z = [\psi(\mathbf{x}_0) \ \psi(\mathbf{x}_1) \ \cdots], \quad Z_+ = [\psi(\mathbf{x}_1) \ \psi(\mathbf{x}_2) \ \cdots], \quad (42)$$

together with the stacked input matrix

$$U = [\mathbf{u}_0 \ \mathbf{u}_1 \ \cdots]. \quad (43)$$

The EDMD regression is then solved as a ridge-regularized least-squares problem:

$$\min_{A,B} \left\| Z_{s,+} - [A \ B] \begin{bmatrix} Z_s \\ U_s \end{bmatrix} \right\|_F^2 + \lambda \|[A \ B]\|_F^2, \quad (44)$$

where λ is a small regularization parameter. In the notebook, the nominal fixed-speed EDMD fit uses

$$\lambda = 10^{-6}. \quad (45)$$

This regression produces the lifted transition matrix A , the input matrix B , and the scaling statistics needed for prediction. During rollout, the procedure is: lift the physical state, standardize the lifted state and input, propagate them linearly, unscale the next lifted state, and reconstruct the physical state using C_{sel} .

For the multi-speed extension, the notebook uses an affine standardized model of the form

$$\mathbf{z}_{s,k+1} \approx A\mathbf{z}_{s,k} + B\mathbf{u}_{s,k} + c, \quad (46)$$

where

$$\mathbf{u}_k = \begin{bmatrix} \delta_k \\ \kappa_k \\ V_x \end{bmatrix}. \quad (47)$$

Because this model must represent a broader operating envelope, the identification problem is more difficult than the fixed-speed case, and the notebook explicitly notes that worse one-step prediction accuracy is expected in this broader setting.

E. Prediction evaluation procedure

The learned EDMD model is evaluated in two stages. First, one-step validation error is computed on held-out trajectories in physical coordinates after reconstructing the next state from the lifted prediction. The report uses root-mean-square error (RMSE) per state, and the notebook also separates raw error from zero-mean residual error to visualize whether there is systematic bias in the predictor. Second, the model is evaluated in multi-step open-loop rollout mode on test trajectories, where the learned predictor is iterated forward over a finite horizon using recorded steering and curvature sequences. This second evaluation is especially important because very small one-step error can still hide accumulated drift over longer horizons. The notebook therefore treats multi-step rollout quality as a more informative test of practical predictive reliability than one-step metrics alone.

F. MPC design using the learned lifted model

The learned EDMD model is embedded into a finite-horizon Model Predictive Control formulation. In the simplest conceptual form, the controller predicts future lifted states using

$$\mathbf{z}_{s,k+1} = A\mathbf{z}_{s,k} + B\mathbf{u}_{s,k}, \quad (48)$$

while physical states are reconstructed by

$$\mathbf{x}_k = C_{\text{sel}}\mathbf{z}_k. \quad (49)$$

The tracking quantities of primary interest are the lateral and heading errors,

$$\mathbf{e}_k = \begin{bmatrix} e_{y,k} \\ e_{\psi,k} \end{bmatrix}, \quad (50)$$

although the full state also appears in the regulation cost used by the implemented controller. The nominal finite-horizon objective in the notebook is quadratic and penalizes state deviation, total steering magnitude, corrective steering effort, and steering-rate variation. In generic form, the cost can be written as

$$\sum_{i=0}^{N-1} (\|\mathbf{x}_i\|_Q^2 + \|\delta_i\|_R^2 + \|\delta_i^{\text{corr}}\|_{R_u}^2 + \|\delta_i - \delta_{i-1}\|_{R_\Delta}^2) + \|\mathbf{x}_N\|_{Q_f}^2. \quad (51)$$

Because the lifted dynamics are linear and the objective is quadratic, the resulting optimization problem is a convex quadratic program. In the notebook it is solved primarily with CVXPY using the OSQP solver, with fallback handling when needed.

The implemented controller uses a *feedforward-centered* formulation. Instead of optimizing the steering angle directly from scratch, the notebook first computes a nominal feedforward steering preview from the reference curvature and then lets MPC optimize only a corrective steering term around that nominal signal:

$$\delta_k = \delta_k^{\text{ff}} + \delta_k^{\text{corr}}. \quad (52)$$

This reduces steady-state bias and makes the optimization focus on correcting residual tracking error rather than rediscovering the obvious curvature-to-steering component of the maneuver. Inside the predictive model, reference curvature is treated as a known preview signal, while steering correction is the main decision variable.

G. Controller constraints and stabilization details

The MPC controller enforces both steering magnitude and steering-rate constraints:

$$|\delta_k| \leq \delta_{\text{max}}, \quad |\delta_k - \delta_{k-1}| \leq \dot{\delta}_{\text{max}} \Delta t. \quad (53)$$

These constraints match the actuator limits already used in the truth-model simulations. A practical numerical issue arises because the learned lifted transition matrix is very close to the unit-circle stability boundary, with spectral radius slightly above one in the nominal EDMD model. To make the optimization more reliable, the notebook applies a minimal spectral shrink to the learned matrix before using it inside MPC. This shrink is not a re-identification step; it is a controller-side stabilization device applied only to improve numerical robustness of the optimization while preserving the learned structure as closely as possible.

H. Nominal control weights and implementation details

The notebook contains two layers of control weights: the baseline configuration object used for the classical controller and the tuned weights used in the final feedforward-centered

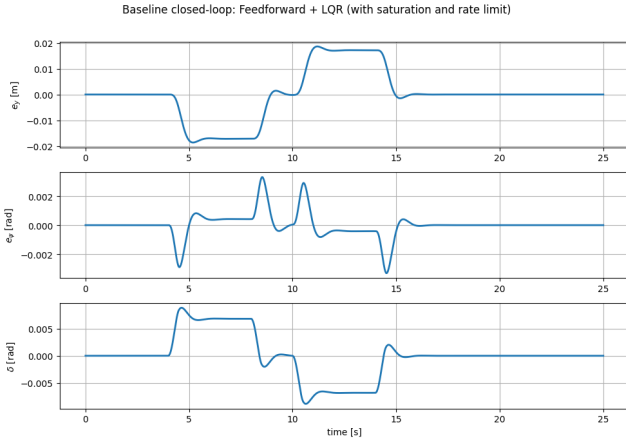


Fig. 3. Closed-loop tracking performance of the baseline feedforward + LQR controller for the lane-change maneuver. The controller maintains small lateral and heading errors while respecting steering saturation and steering-rate limits.

Koopman-MPC formulation. In the general configuration, the nominal values are

$$q_{e_y} = 50, \quad q_{e_\psi} = 50, \quad q_{v_y} = 10, \quad q_r = 10, \quad (54)$$

$$r_\delta = 1, \quad r_{\Delta\delta} = 10. \quad (55)$$

In the final Koopman-MPC implementation, stronger output-tracking emphasis is used together with separate penalties on total steering, corrective steering, and steering-rate variation. The controller is implemented in Python using `cvxpy` for optimization, with `OSQP` as the primary quadratic-program solver and `SCS` as a fallback. The overall notebook is config-driven and artifact-producing: learned models, figures, tables, and run manifests are automatically saved to structured results directories so that the full experimental workflow can be reproduced.

I. Baseline controller

A classical feedforward plus LQR controller is used as the main control baseline. The feedforward term is obtained from a practical curvature-to-steering relation with an understeer-gradient correction, while the feedback term is a discrete-time LQR law designed from a linearized bicycle model at the nominal operating speed. The total steering command is

$$\delta = \delta_{ff} + \delta_{fb}, \quad \delta_{fb} = -K\mathbf{x}. \quad (56)$$

The same steering saturation and steering-rate constraints used in the learned-controller experiments are also enforced here. This baseline is intentionally strong rather than trivial, since the project aims to compare Koopman-MPC against a meaningful classical benchmark rather than against an artificially weak reference controller.

J. SINDy predictor and comparison model

In addition to Koopman/EDMD, the project fits a SINDy-style discrete-time predictor from the simulated trajectory data. The SINDy model is trained using trajectory-consistent state-input pairs, so that end-of-trajectory samples are never

TABLE I
NOMINAL CLOSED-LOOP TRACKING METRICS FOR BASELINE LQR AND KOOPMAN-MPC.

Metric	Lane-BL	Lane-KMPC	Slalom-BL	Slalom-KMPC
RMS e_y [m]	0.009534	0.037620	0.012183	0.037619
Max $ e_y $ [m]	0.018609	0.039390	0.017238	0.039362
RMS e_ψ [rad]	0.000813	0.001108	0.000842	0.001098
Max $ e_\psi $ [rad]	0.00332	0.01168	0.00120	0.01139

incorrectly connected to the beginning of another rollout. A structured candidate library is built from constants, states, inputs, quadratic terms, cross terms, and trigonometric features. The notebook reports a library of 30 candidate terms and fits a sparse model using sequential thresholded least squares. The resulting predictor converges to only 3 active terms per state equation, yielding a compact nonlinear model. In this project, SINDy is used as a *prediction benchmark* rather than as the controller model. This distinction is important, because the final report compares SINDy and EDMD primarily in terms of open-loop predictive quality, whereas the closed-loop controller comparison focuses on baseline LQR versus Koopman-MPC.

K. Why this methodology is appropriate for the project

This methodology is appropriate because it directly supports the main comparative question of the report. The truth-model simulator provides a controlled environment. The data pipeline creates informative but reproducible trajectories. EDMD supplies a learned linear structure suitable for MPC. SINDy provides a competing learned predictor with different modeling bias. The baseline controller provides a strong classical reference. Together, these elements allow the project to examine not only whether a learned model can predict well, but whether that learned structure leads to superior control performance and operating-point robustness. That is the central scientific and engineering objective of the study.

V. RESULTS

A. Nominal closed-loop tracking

The first question in the study is whether the learned Koopman-MPC improves closed-loop tracking relative to the classical feedforward plus LQR baseline under nominal operating conditions. The notebook evaluates both controllers on the same truth-model plant for two representative reference maneuvers: lane change and slalom. Table I summarizes the main tracking metrics.

The nominal comparison is clear and consistent across both maneuvers. The baseline feedforward plus LQR controller achieves smaller RMS lateral error and smaller RMS heading error than Koopman-MPC for both lane-change and slalom tracking. The notebook also notes that Koopman-MPC remains feasible and stable, but exhibits a persistent lateral offset and less smooth steering behavior than the baseline. Therefore, the learned lifted model is sufficiently accurate to support a working controller, yet the resulting closed-loop performance still does not surpass the classical benchmark. This is already

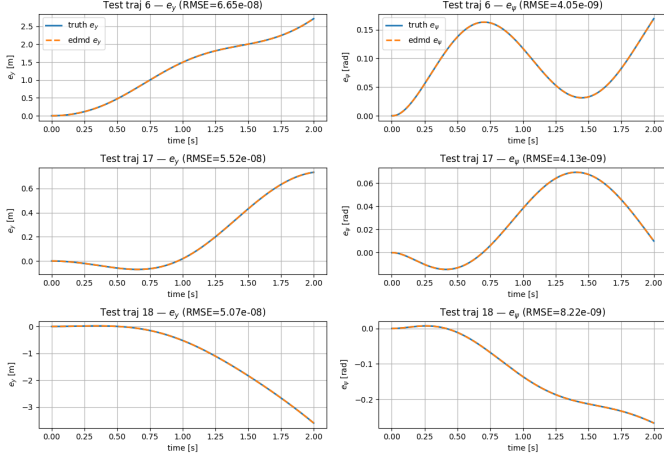


Fig. 4. Multi-step open-loop prediction on held-out test trajectories using the learned EDMD model. The predicted lateral and heading errors closely match the ground-truth rollouts over the 200-step horizon, indicating excellent finite-horizon predictive accuracy in the nominal operating regime.

an important project result: a controller-friendly learned model can be viable without being the best controller in practice.

B. Open-loop predictive performance of the fixed-speed EDMD model

Before interpreting the controller results, it is important to note that the fixed-speed EDMD model performs extremely well as a predictor in the nominal operating regime. On the validation set, the notebook reports one-step RMSE values of approximately

$$[1.59 \times 10^{-9}, 7.37 \times 10^{-10}, 2.74 \times 10^{-12}, 2.40 \times 10^{-11}] \quad (57)$$

for $[e_y, e_\psi, v_y, r]$, respectively. In multi-step open-loop rollout over a 200-step horizon on held-out trajectories, the mean RMSE remains extremely small:

$$[5.75 \times 10^{-8}, 5.47 \times 10^{-9}, 4.94 \times 10^{-10}, 3.09 \times 10^{-10}]. \quad (58)$$

Thus, the fixed-speed EDMD model is highly accurate in open loop within the nominal regime, even though the associated Koopman-MPC does not outperform the classical controller in closed loop. This contrast is one of the central findings of the report.

C. Fixed-speed robustness sweep

To evaluate generalization beyond the nominal operating point, the notebook performs a speed sweep over

$$V_x \in \{10, 15, 20, 25\} \text{ m/s} \quad (59)$$

for both lane-change and slalom maneuvers. This experiment is especially important because the original Koopman/EDMD model is learned around a single nominal speed, so the sweep reveals how well the learned-controller design transfers as the vehicle dynamics change with speed.

For the fixed-speed Koopman-MPC, the reported RMS lateral errors are:

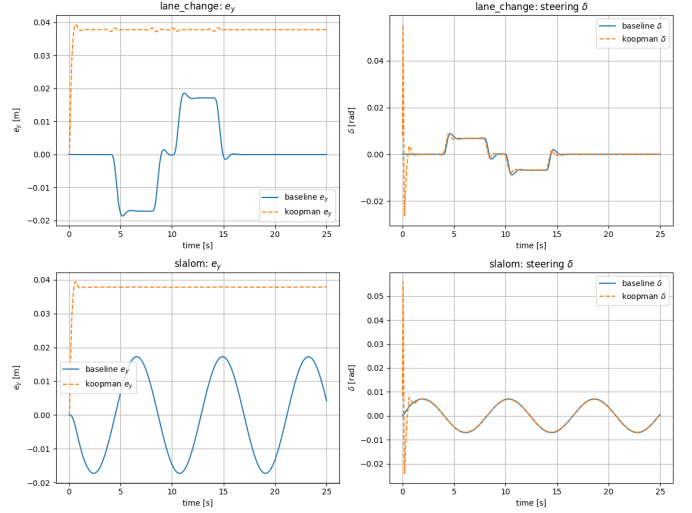


Fig. 5. Closed-loop comparison of baseline feedforward + LQR and Koopman-MPC on the nominal lane-change and slalom maneuvers. Across both maneuvers, the baseline controller achieves lower lateral tracking error, while Koopman-MPC exhibits a persistent offset despite remaining feasible and stable in the nominal regime.

- **Lane change:** 0.0371, 0.0356, 0.0349, and 82.75 m at $V_x = 10, 15, 20, 25$ m/s,
- **Slalom:** 0.0357, 0.0356, 0.0358, and 70.07 m at $V_x = 10, 15, 20, 25$ m/s.

The corresponding baseline RMS lateral errors are much smaller:

- **Lane change:** 0.00158, 0.01108, 0.03311, and 0.07343 m,
- **Slalom:** 0.00170, 0.01200, 0.03583, and 0.07939 m.

These numbers show that the baseline controller remains strong across the entire tested speed range, whereas the fixed-speed Koopman-MPC is consistently worse and degrades catastrophically at the highest speed.

In addition to the aggregate RMSE values, the failure behavior can be interpreted from the trajectory evolution. At higher speeds, the lateral error does not increase gradually but diverges rapidly after an initial period of tracking. This indicates that the controller initially operates within a locally valid regime of the learned model, but exits that regime as the prediction horizon evolves. The resulting model mismatch leads to incorrect steering corrections that accumulate over time and produce large tracking errors.

In fact, the notebook introduces a practical failure rule based on $\text{RMS}(e_y) > 2.0$ m and reports Koopman-MPC failure at 25 m/s for both lane-change and slalom. The engineering interpretation is that the fixed-speed learned controller has limited operating-point robustness: it remains usable over a narrow region, but it is not reliable as speed moves significantly away from the nominal training condition.

D. Speed-aware EDMD and vx-aware Koopman-MPC

Because the fixed-speed controller generalizes poorly, the project next explores a speed-aware EDMD extension. A

Robustness sweep: RMS lateral error vs speed

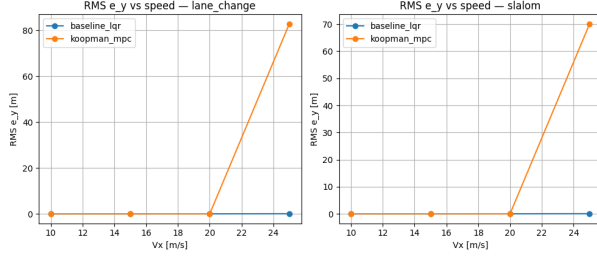


Fig. 6. Robustness sweep across longitudinal speed for baseline LQR and fixed-speed Koopman-MPC. The baseline controller remains stable across the tested range, whereas the fixed-speed Koopman-MPC exhibits substantially larger lateral tracking error and fails at the highest tested speed.

multi-speed dataset is constructed using longitudinal speeds $\{10, 15, 20, 25\}$ m/s. The notebook reports a total of 40 trajectories, split into 28 training, 6 validation, and 6 test rollouts. The learned speed-aware model preserves the same lifted dimension,

$$n_z = 17, \quad (60)$$

but augments the input to

$$\mathbf{u}_k = \begin{bmatrix} \delta_k \\ \kappa_k \\ V_x \end{bmatrix}. \quad (61)$$

Its reported one-step validation RMSE is

$$[0.01153149, 0.00012162, 0.00989414, 0.00441424] \quad (62)$$

for $[e_y, e_\psi, v_y, r]$. This is notably worse than the fixed-speed EDMD one-step validation error, which is expected because a single affine lifted model is now required to represent dynamics across a broader operating envelope.

This degradation can be understood from a modeling perspective. By augmenting the input with V_x , the learned predictor must approximate parameter-varying dynamics across multiple operating conditions using a fixed linear representation in lifted coordinates. This increases the approximation burden and forces the model to trade local accuracy at the nominal operating point for broader coverage, resulting in higher prediction error.

The critical question, however, is whether this broader predictor improves closed-loop control. The answer in the notebook is no. The speed-aware Koopman-MPC produces substantially worse RMS lateral error than the baseline controller at every tested speed and for both maneuvers. For lane change, the reported RMS lateral errors are

$$5.362674, 25.839373, 58.166768, 117.840062 \text{ m}, \quad (63)$$

and for slalom they are

$$5.373165, 29.790355, 66.297425, 129.514152 \text{ m}, \quad (64)$$

at $V_x = 10, 15, 20, 25$ m/s, respectively. These values are dramatically larger than the corresponding baseline values.

Speed sweep: Baseline vs Speed-aware Koopman-MPC

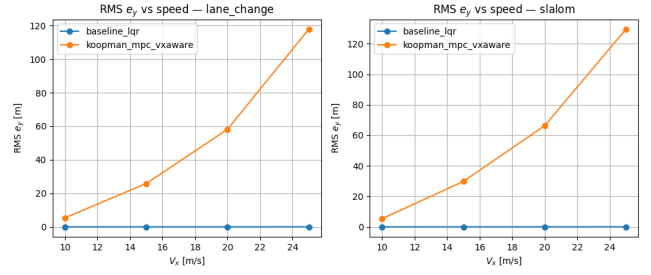


Fig. 7. Speed sweep comparison between the baseline controller and the speed-aware Koopman-MPC attempt. The vx-aware learned controller does not improve robustness and produces substantially larger lateral tracking error across the tested speed range.

Speed sweep (linear): Baseline vs Koopman-fixed vs Koopman-vxaware

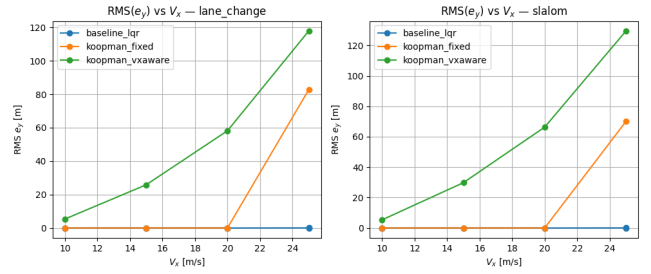


Fig. 8. Final robustness comparison across longitudinal speed for the baseline controller, fixed-speed Koopman-MPC, and speed-aware Koopman-MPC. The baseline feedforward + LQR controller achieves the best overall performance across both maneuvers, while the fixed-speed Koopman-MPC degrades severely at high speed and the vx-aware extension performs even worse over the tested range.

The poor closed-loop performance can be explained by error propagation within the MPC framework. The increased prediction error of the speed-aware model is compounded over the prediction horizon, leading to inaccurate future-state estimates and incorrect steering corrections. As a result, the controller produces large and inconsistent control actions, which cause the tracking error to grow rapidly.

This negative result is still informative: it shows that simply adding speed as an extra input is not sufficient to obtain a robust speed-aware Koopman-MPC controller. Instead, achieving robustness across operating conditions likely requires more structured model representations or controller architectures, such as explicitly parameterized LPV models or gain-scheduled control designs.

E. Predictor comparison: Koopman/EDMD versus SINDy

The notebook concludes with a direct comparison between the two learned predictors, Koopman/EDMD and SINDy, using both one-step validation RMSE and multi-step open-loop rollout RMSE for the physical states $[e_y, e_\psi, v_y, r]$. The final comparison reports that SINDy outperforms Koopman/EDMD for all four states in both metrics. This is a strong result because both predictors are trained on the same simulated bicycle-model dataset and evaluated on held-out trajectories.

Multi-step open-loop prediction: Truth vs SINDy (test set) | horizon=200 steps

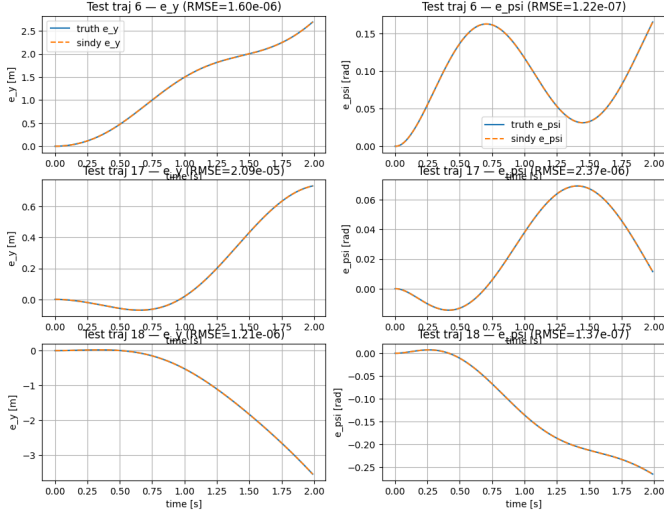


Fig. 9. Multi-step open-loop prediction on held-out test trajectories using the learned SINDy model. The predicted trajectories closely track the ground-truth rollouts over the 200-step horizon, confirming that SINDy also achieves excellent predictive accuracy on the simulated dataset.

For SINDy, the reported one-step validation RMSE is approximately

$$[3.13 \times 10^{-10}, 1.46 \times 10^{-8}, 1.67 \times 10^{-8}, 4.82 \times 10^{-9}], \quad (65)$$

while the final predictor-comparison table reports Koopman one-step RMSE values of approximately

$$[1.915 \times 10^{-3}, 1.419 \times 10^{-5}, 3.015 \times 10^{-8}, 1.452 \times 10^{-5}]. \quad (66)$$

For multi-step rollout RMSE, the same table shows Koopman values of approximately

$$[2.41491 \times 10^{-1}, 1.825 \times 10^{-3}, 3.01 \times 10^{-4}, 1.66 \times 10^{-4}], \quad (67)$$

whereas SINDy achieves much smaller values, approximately

$$[7.88 \times 10^{-6}, 8.77 \times 10^{-7}, 1.56 \times 10^{-7}, 7.30 \times 10^{-8}]. \quad (68)$$

Thus, on this deterministic and noise-free simulated dataset, SINDy is the strongest learned predictor in pure open-loop accuracy. Koopman/EDMD remains valuable, however, because it yields a lifted linear representation that is directly compatible with MPC. That distinction between *best predictor* and *most controller-friendly representation* is the main technical message of the project.

F. Failure Analysis of Koopman-MPC

The failure of Koopman-MPC at higher speeds can be attributed to several interacting factors. First, the EDMD model is trained using data generated at a nominal speed of $V_x = 15$ m/s, while the robustness evaluation includes speeds up to 25 m/s. This creates a distribution shift, forcing the learned model to extrapolate beyond the training regime. Second, the fixed lifting dictionary is not explicitly designed

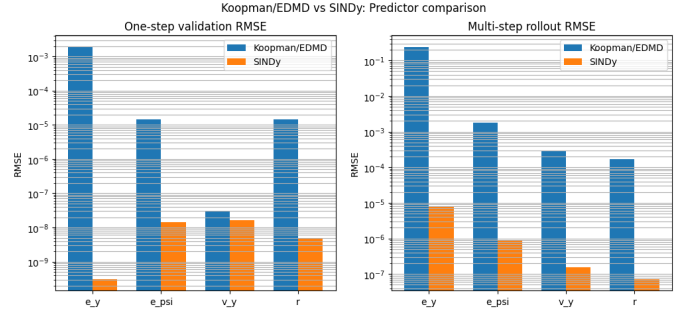


Fig. 10. Predictor comparison between Koopman/EDMD and SINDy. SINDy achieves lower one-step validation RMSE and lower multi-step rollout RMSE across the reported physical states, while Koopman/EDMD remains the more convenient model structure for convex MPC synthesis.

to capture the parameter-varying nature of the dynamics with respect to speed, leading to structural model mismatch at higher operating conditions. Finally, the MPC formulation is sensitive to model inaccuracies, and even small prediction errors can accumulate over the prediction horizon, resulting in incorrect control actions. These effects together explain the sharp degradation in closed-loop performance observed at higher speeds. This behavior is consistent with the observed results, where RMS lateral error increases dramatically at $V_x = 25$ m/s despite near-perfect one-step prediction accuracy at the nominal speed.

VI. COMPARISON AND ABLATION STUDY

This project contains two comparison axes: controller comparison and predictor comparison. Together, they form an ablation-style analysis because they separate three related questions: whether the limitation comes from the controller architecture, the learned prediction model, or the operating-region generalization of the model-controller pair.

A. Controller Comparison

On the controller side, the three most important cases are

- 1) baseline feedforward plus LQR,
- 2) fixed-speed Koopman-MPC,
- 3) speed-aware Koopman-MPC.

The final comparison across maneuvers and speeds is decisive. The classical baseline is the best overall controller across the tested speeds and for both lane-change and slalom maneuvers. The fixed-speed Koopman-MPC does not surpass the baseline and degrades severely at 25 m/s, while the speed-aware Koopman-MPC also remains weaker than the baseline across the sweep. This means that the central limitation is not only the omission of speed from the fixed-speed model. Even after augmenting the predictor with speed, the learned controller remains inferior in closed loop. The practical lesson is that robust learned control requires more than improved predictor coverage alone; it also requires a controller design and model structure that remain reliable under operating-point variation.

Final comparison: prediction accuracy vs control performance

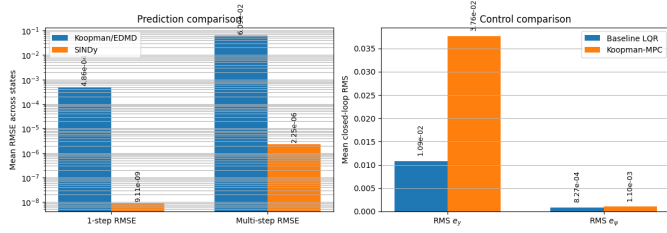


Fig. 11. Final comparison between prediction accuracy and control performance. On the prediction side, SINDy achieves lower mean one-step and multi-step RMSE than Koopman/EDMD. On the control side, the baseline LQR controller achieves substantially lower closed-loop RMS tracking error than Koopman-MPC. This summarizes the central lesson of the project: the best predictor is not necessarily the best controller.

B. Predictor Comparison

On the predictor side, the comparison is between EDMD and SINDy. Here, the conclusion is almost the opposite of the controller comparison. SINDy is the stronger predictor in one-step and multi-step open-loop accuracy for all four physical states, while Koopman/EDMD is the more convenient representation for controller synthesis because it provides a lifted linear model suitable for convex MPC. This directly supports the report’s central claim:

$$\text{prediction accuracy} \neq \text{controller quality}. \quad (69)$$

In this project, the model with the strongest predictive accuracy is not the one used for the strongest controller, and the learned representation most naturally suited for MPC is not the one that dominates the open-loop RMSE comparison. That separation is what makes the study interesting from a data-driven control perspective.

C. Ablation and Comparison Summary

Table II summarizes the main modeling and control variants considered in the project.

The ablation summary reinforces the main result of the project. Adding reference curvature makes the EDMD model more appropriate for path-tracking MPC than a reduced-input predictor, and adding longitudinal speed is a reasonable attempt to improve robustness across operating points. However, neither learned-control variant surpasses the classical feedforward plus LQR baseline in closed-loop tracking. Therefore, the study shows that improved predictor structure and improved controller performance are related but not equivalent objectives.

VII. DISCUSSION

The central engineering lesson of this project is that prediction accuracy, controller-oriented model structure, and closed-loop robustness are fundamentally different objectives, and they do not necessarily align with the same method. This distinction became clear only because the notebook compared the full pipeline rather than stopping at one-step prediction accuracy. Koopman/EDMD was attractive because it produced

a learned lifted linear predictor that fit naturally into a convex MPC formulation. That structural advantage is real and is one of the main reasons Koopman methods are appealing in data-driven control. However, the final closed-loop results show that controller-friendly structure alone does not guarantee the best tracking performance. In this study, the resulting Koopman-MPC remained feasible and stable in nominal conditions, but it did not outperform the classical baseline controller.

The baseline feedforward plus LQR controller emerged as the strongest tested controller because it remained closely aligned with the known plant physics and maintained good performance across both nominal maneuvers and speed-varying tests. At nominal speed, it achieved lower RMS lateral and heading error than Koopman-MPC, and across the full speed sweep it remained the best overall controller for both lane-change and slalom maneuvers. By contrast, the fixed-speed Koopman controller degraded substantially as speed increased and failed badly at the highest tested operating point. This behavior suggests that the learned controller was much more sensitive to operating-point shift and training-data coverage than the classical baseline. In other words, the learned model was accurate enough to be useful locally, but not robust enough to maintain reliable closed-loop behavior across the broader tested envelope.

The speed-aware extension sharpened this conclusion further. Adding longitudinal speed as an additional EDMD input was a reasonable and well-motivated attempt to improve operating-point robustness, but in practice it did not solve the problem. The V_x -aware Koopman-MPC produced substantially worse closed-loop tracking than the baseline across the tested speed range and therefore should be interpreted as a negative but informative result. This matters because it shows that the weakness of the learned controller was not merely the omission of speed as an explicit variable. More broadly, it indicates that robust learned control may require richer structure than simple input augmentation, such as better model regularization, stronger stabilization, or a more explicitly structured LPV or gain-scheduled design.

The SINDy comparison makes the overall message even clearer. On this deterministic, noise-free simulated dataset, SINDy achieved lower one-step and lower multi-step open-loop error than Koopman/EDMD for all four physical states. Thus, in pure prediction terms, SINDy was the strongest learned model in the notebook. Yet SINDy was not the main controller model because its sparse nonlinear form was less convenient to place directly inside the convex MPC formulation used here. Koopman/EDMD, while weaker as a pure predictor, remained valuable because it yielded a learned model structure that was directly compatible with constrained optimization-based control. This is the main systems-level insight of the project: the best predictor is not necessarily the best controller model, and the model that is most convenient for controller synthesis is not necessarily the one that minimizes open-loop prediction error.

Taken together, the results show that data-driven vehicle lateral control is feasible and informative, but method selection

TABLE II
ABLATION-STYLE SUMMARY OF MODELING AND CONTROL VARIANTS.

Variant	Purpose	Main observation
EDMD without explicit κ_{ref} input	Reduced-input ablation/reference case	Less suitable for changing-path prediction because reference curvature directly enters the tracking-error dynamics.
EDMD with δ and κ_{ref} inputs	Main Koopman-MPC predictor	Provides a controller-compatible lifted linear model for constrained MPC.
Speed-aware EDMD with V_x augmentation	Robustness extension	Includes speed variation, but still does not outperform the baseline controller.
SINDy predictor	Open-loop prediction benchmark	Achieves the strongest one-step and multi-step prediction accuracy.
Feedforward plus LQR	Classical control baseline	Achieves the best closed-loop tracking and robustness across the tested maneuvers.

depends strongly on the engineering objective. If the goal is the best tested closed-loop controller on this problem, the baseline feedforward plus LQR design is the best choice. If the goal is pure prediction accuracy on the simulated dataset, SINDy is the strongest learned model. If the goal is to obtain a learned model structure that fits naturally inside MPC, Koopman/EDMD remains the most useful representation among the methods tested here. That separation among objectives is the most important conceptual outcome of the report.

VIII. LIMITATIONS AND FUTURE WORK

Several limitations should be kept in mind when interpreting the results of this study. First, the entire project is simulation-based and deterministic. This is valuable for isolating the behavior of the learned models and controllers, but it also means the methods were not tested against measurement noise, sensor latency, unmodeled tire behavior, actuator delay, or parameter drift. As a result, the reported prediction and control performance should be interpreted as results in a clean and controlled environment rather than as a claim of immediate real-world robustness. The notebook itself notes that the simulated, noise-free setting strongly favors pure predictive methods such as SINDy, especially when the feature library is well matched to the underlying dynamics.

Second, the ground-truth plant is itself an idealized dynamic bicycle model with linear tire forces under a small-slip approximation. This is an appropriate benchmark model for a data-driven dynamical-systems project, but it is still a simplified representation of true vehicle lateral dynamics. Therefore, the study should be interpreted as a controlled proof-of-concept and comparative investigation rather than as a high-fidelity vehicle-control validation. Relatedly, the learned models were trained on data generated by the same model used for evaluation, so the project examines generalization across operating conditions within one simulated world rather than across fundamentally different plant classes.

Third, the Koopman lifting design is fixed and relatively simple. The observable dictionary includes physically meaningful states together with polynomial, cross, and trigonometric features, but it is not the only possible choice, nor is it guaranteed to be the best one for robust control. A richer lifting dictionary, a more structured set of observables, or a better regularized lifted model may provide a stronger compromise between predictive fidelity and controller usefulness. The

notebook explicitly identifies richer Koopman dictionaries and improved regularization and stabilization of lifted models as natural future directions.

Fourth, the speed-aware extension was intentionally modest: it augments the EDMD input with V_x , but it does not impose a structured LPV form, gain-scheduled controller architecture, or deeper coupling between operating condition and controller synthesis. The poor performance of the V_x -aware Koopman-MPC therefore should not be interpreted as evidence that speed-aware learned control is impossible. Rather, it shows that *this specific extension* was not sufficient. The notebook explicitly frames this as a negative but informative result and suggests that a successful speed-aware controller will likely require more than simple input augmentation alone.

These limitations point naturally to several directions for future work. Promising next steps include richer Koopman lifting dictionaries, improved regularization and stabilization of learned lifted models, larger multi-speed datasets, structured LPV or gain-scheduled Koopman formulations, and hybrid physics-informed plus data-driven learning strategies. Beyond model design, an especially important next step would be to build a speed-aware *controller architecture*, not only a speed-aware predictor, so that operating-condition variation is handled explicitly during control synthesis. Additional extensions could also include noisy simulation studies, disturbance injection, and evaluation on real or experimentally collected vehicle datasets. Such directions would test whether the conclusions of this project remain valid when the clean assumptions of the present study are relaxed.

A. Ablation and Model Sensitivity Considerations

While the project compares different modeling and control approaches, it does not include a detailed ablation study within the Koopman/EDMD framework itself. In particular, key design choices such as the lifting dictionary, regularization strength, and MPC prediction horizon were fixed throughout the study. As a result, the observed performance reflects a single configuration of Koopman-MPC rather than the full range of possible design trade-offs.

First, the lifting dictionary was fixed to a predefined set of polynomial, cross, and trigonometric features. A simple but informative extension would be to perform a lifting-dictionary ablation study. For example, one could compare the current dictionary against reduced and expanded variants, such as

removing trigonometric terms or adding higher-order nonlinear features. Evaluating the resulting predictors in terms of one-step RMSE, multi-step rollout error, and closed-loop tracking performance would reveal how sensitive Koopman-MPC is to the choice of observables. The choice of observables directly determines how well nonlinear dynamics can be represented in the lifted linear space. Alternative dictionaries, including richer nonlinear features or structured basis functions, may improve predictive accuracy and robustness across operating conditions, but may also increase model complexity and risk of overfitting.

Second, the ridge regularization parameter used in EDMD was not varied. A complementary extension would be to perform a joint sensitivity study over the regularization parameter and MPC horizon length. For instance, evaluating a small grid of values for λ and prediction horizon N would help identify the trade-off between numerical stability, prediction accuracy, and closed-loop robustness. Such an analysis would clarify whether the observed performance degradation is primarily due to model approximation error or its amplification within the MPC optimization. This parameter controls the trade-off between numerical stability and model fidelity. Exploring different values could reveal how sensitive the learned predictor and resulting controller are to estimation noise and model complexity, and different choices may significantly affect generalization performance.

Third, the MPC design parameters, particularly the prediction horizon, were not systematically varied. The horizon length directly influences how model inaccuracies propagate during optimization: shorter horizons limit error accumulation, while longer horizons rely more heavily on accurate multi-step prediction. This introduces a trade-off between prediction horizon length and robustness to model mismatch in closed-loop control.

Taken together, these considerations show that Koopman-MPC performance depends not only on the learned model, but on the interaction between feature design, regularization, and control design choices, which were not systematically explored in this study.

More broadly, this study highlights that in data-driven control, model design must be evaluated not only by predictive accuracy, but by its role within the feedback loop, where robustness and structural compatibility ultimately determine closed-loop performance.

IX. CONCLUSION

This project developed and evaluated a complete data-driven lateral vehicle modeling and control pipeline based on Koopman-inspired EDMD and Model Predictive Control. Using a simulated dynamic bicycle model in road-aligned tracking-error coordinates, the study generated informative rollouts, learned lifted linear predictors, and tested those predictors in both open-loop prediction and closed-loop control settings.

The learned Koopman/EDMD model provided an accurate and controller-friendly lifted representation in the nominal

operating regime, and the resulting Koopman-MPC controller remained feasible and stable under nominal maneuvers. However, the closed-loop results showed that the classical feedforward plus LQR baseline achieved better tracking accuracy and substantially stronger robustness across the tested speed range. In parallel, SINDy emerged as the strongest learned predictor in open-loop error metrics, outperforming Koopman/EDMD in both one-step and multi-step prediction.

The main conclusion of the project is therefore clear: in data-driven control, the best predictor does not necessarily produce the best controller. SINDy was strongest for pure prediction, Koopman/EDMD was most useful as an MPC-compatible learned representation, and feedforward plus LQR remained the strongest closed-loop controller for the tested vehicle lateral-control problem. Successful feedback design therefore depends not only on prediction accuracy, but also on model structure, operating-regime coverage, and robustness of the learned model when embedded in closed-loop control.

ACKNOWLEDGMENT

Grateful acknowledgment is extended to Prof. Amarsagar Reddy Ramapuram Matavalam for guidance and insightful discussions in EEE 598: Data Driven Dynamical System Analysis and Control. The course framework and feedback played a significant role in shaping this project.

REFERENCES

- [1] A. Mishra (aymisxx), "Vehicle Lateral Stability (Python Implementation)," GitHub repository, 2026. [Online]. Available: <https://github.com/aymisxx/VehicleLateralStability-Py>
- [2] I. Mezić, "Spectral properties of dynamical systems, model reduction and decompositions," *Nonlinear Dynamics*, vol. 41, no. 1–3, pp. 309–325, 2005.
- [3] M. O. Williams, I. G. Kevrekidis, and C. W. Rowley, "A data-driven approximation of the Koopman operator: Extending dynamic mode decomposition," *Journal of Nonlinear Science*, vol. 25, no. 6, pp. 1307–1346, 2015.
- [4] S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Discovering governing equations from data by sparse identification of nonlinear dynamical systems," *Proceedings of the National Academy of Sciences*, vol. 113, no. 15, pp. 3932–3937, 2016.
- [5] S. L. Brunton and J. N. Kutz, *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*. Cambridge, U.K.: Cambridge University Press, 2019.
- [6] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Nob Hill Publishing, 2017.